جلسهٔ ۳ از ۵

مهارت‌های عامل

هر عامل (agent) عمومی را به‌اقتضا به متخصص بدل کن
بدون تورم زمینه، قابل‌حمل و سبک

بر پایهٔ مقالهٔ تانوی سینگال، گابریلا هرناندز لاریوس، دبانشو دوس، لاوی نیگام و اسمیتا کولان

پل از جلسات پیشین

کجای نقشه ایستاده‌ایم؟

جلسهٔ ۱

مهار (harness): هرچه مدل را احاطه می‌کند

←

جلسهٔ ۲

ابزار (tool): عامل چگونه به جهان وصل می‌شود

←

جلسهٔ ۳

دانش رویه‌ای: عامل چگونه می‌داند **چطور** کاری را انجام دهد

تعریف ساده: مهارت (skill) عامل، پوشه‌ای است شامل یک `SKILL.md` و به‌اختیار پوشه‌های `scripts/` ، `references/` و `assets/` .

چهار اصطکاکی که مهارت‌ها حل کردند

۱

دستور زیاد، نتیجهٔ بدتر

ریختن هر دستوری که به ذهنتان می‌رسد در یک پرامپت (prompt) سامانه، عملکرد مدل را خراب می‌کند. مهارت‌ها فقط هنگام نیاز بارگذاری می‌شوند.

۲

دانستن «چگونه»، نه فقط «چه»

مدل‌ها معادل‌هایی برای حافظهٔ رویدادی و معنایی دارند؛ آنچه نداشتند، راهی برای به‌خاطرسپردن **گام‌به‌گام انجام‌دادن** بود.

۳

اضافه‌بار چندعاملی

ساخت و نگه‌داری سامانه‌های چندعاملیِ پیچیده دشوار است. مهارت‌ها اجازه می‌دهند یک عامل عمومی روان در نقش‌های تخصصی فرو برود.

۴

قابلیت حمل

یک پوشه با یک فایل مارک‌داون، اولیه‌ای بسیار سبک است. هر عاملی که به فایل‌سیستم دسترسی دارد می‌تواند استفاده کند.

مهارت عامل، نخستین اولیهٔ باورپذیر حافظهٔ رویه‌ای (Procedural memory) برای عامل‌های مدل‌زبانی است.

حافظهٔ رویدادی: چه گذشت · حافظهٔ معنایی: چه می‌دانیم · حافظهٔ رویه‌ای: چطور انجامش بدهیم

نقشهٔ امروز: دو مخاطب

سازندگان — نیمهٔ اول

مهارت چیست، چطور می‌سازیمش، چطور نصبش می‌کنیم، و چه تفاوتی با MCP و `AGENTS.md` دارد.

توسعه‌دهندگان — نیمهٔ دوم

ارزیابی، آمادگی تولید، فرامهارت‌ها، ترکیب و بسته‌بندی، و انتخاب از میان هزاران مهارت موجود.

بخش یک

مهارت چیست و چطور نخستین‌اش را بسازیم؟

بله، یک مهارت می‌تواند فقط یک فایل مارک‌داون باشد. اما لازم نیست همان‌جا بماند.

دو مسیر پیدایش مهارت

مسیر الف — ترجمهٔ آنچه از قبل می‌دانید

متخصصان موضوعی که دانش نهادی‌شان جایی نوشته شده: افسر انطباق با یک دفترچهٔ سی‌صفحه‌ای، مدیر منابع انسانی با راهنمای آموزش تازه‌واردها. لازم نیست برنامه‌نویسی یاد بگیرند؛ محتوا را دارند، فقط باید به قالبی ترجمه شود که عامل هوشمندانه به کارش ببرد.

مسیر ب — تبلور آنچه عامل همین حالا انجام داد

عامل یک کار تکراری و غیرپیش‌پاافتاده را با موفقیت انجام داد. نمی‌خواهید دفعهٔ بعد از صفر کشفش کند. پس می‌خواهید عامل از همان اجرای موفق، مهارتی بسازد.

الگوی در حال شکل‌گیری: هر چیزی که گردش‌کار (workflow) خوب و قابل‌استفادهٔ مجدد باشد، مهارت می‌شود — و لازم نیست خودتان بنویسیدش: عامل می‌نویسد، شما بازبینی می‌کنید.

کالبدشکافی یک مهارت

```
cafe-preparation/
├── SKILL.md                    # Required: metadata + instructions
├── scripts/                    # Optional: executable code (any language)
│   ├── calc_quantities.py      # Estimates lattes, croissants, etc.
│   └── convert_to_ingredients.py
├── references/                 # Optional: supplementary context
│   ├── menu_and_recipes.md     # Matcha latte: 3g powder, 200ml ...
│   └── minimums.md             # Always prep for at least 40 lattes ...
└── assets/                     # Optional: templates, configs, schemas
    ├── prep_sheet_template.md  # Final layout for the kitchen staff
    └── shopping_list_template.md
```

تنها فایل اجباری `SKILL.md` **است.** بقیه اختیاری‌اند و هر کدام نقش مشخصی در افشای تدریجی (Progressive disclosure) دارند.

افشای تدریجی: سه سطح بارگذاری

۱

فراداده

نام و توصیف، **همیشه** در زمینهٔ عامل حاضرند. حدود ۳۰ تا ۸۰ توکن (token) برای هر مهارت.

۲

بدنهٔ `SKILL.md`

فقط زمانی بارگذاری می‌شود که مهارت فعال شود.

۳

منابع همراه

دقیقاً هنگام نیاز؛ و اسکریپت‌ها اجرا می‌شوند بی‌آنکه پنجرهٔ توکن را آلوده کنند.

نتیجه: می‌توانید صد مهارت نصب‌شده داشته باشید و در هر نوبت (turn) فقط هزینهٔ ناچیز فرادادهٔ آن‌ها را بپردازید.

قلب مهارت: پیشانی‌نوشت

```
---
name: cafe-preparation
description: |
  Calculates daily ingredient needs and generates prep sheets
  for cafe operations.
  Use when the user asks to estimate daily quantities, convert
  drinks to ingredients, or generate shopping lists.
  Do NOT use for employee shift scheduling or financial accounting.
---
```

این متن، ماشهٔ فعال‌سازی است. تنها چیزی که مدل هنگام تصمیم‌گیری برای بارگذاری مهارت می‌بیند، همین است.

دو چیزی که باید از همان اول درست باشد

نام‌گذاری: بدیهی و کسل‌کننده

برای پوشه‌ها `snake_case`، برای نام مهارت `kebab-case`، و ترجیحاً شکل مصدری: `managing-databases` نه `database-manager`.

از نام‌های عمومی مثل `utils` و `tools` و از اصطلاحات داخلی سازمان بپرهیزید.

توصیف: الگوریتم مسیریابی شما

بگویید چه می‌کند، کلیدواژه‌های محرک (trigger) را جلو بیندازید، اگر مدل کم فعال می‌کند سمج باشید، و صریح بگویید **برای چه کاری نیست**.

بیشترین وقت طراحی‌تان باید اینجا صرف شود.

بقیهٔ پوشه: هر چیزی سر جای خودش

- **`scripts/` — کار قطعی (deterministic).**

  تجزیهٔ خروجی‌ها، ریاضیات، قالب‌بندی. مدل تصمیم می‌گیرد چه کند؛ اسکریپت کار سنگین را انجام می‌دهد.

- **`references/` — دانشی که فقط پس از فعال‌شدن مهارت لازم می‌شود.**

  اصول حوزه، تعریف‌ها، رسیدگی به حالت‌های مرزی؛ هنگام نیاز بارگذاری می‌شود.

- **`assets/` — قالب‌ها و اسکیماها (schema).**

قاعدهٔ سرانگشتی: اگر `SKILL.md` دارد بلند می‌شود، پاراگراف بعدی احتمالاً جایش در `references/` است، نه در بدنه.

نصب: قالب مشترک است، مسیر نه

۱. انداختن فایل

عامل‌های کدنویسی و خط فرمان: پوشهٔ مهارت را در دایرکتوری مشخصی می‌اندازید و عامل برش می‌دارد. قراردادی مشترک در حال شکل‌گیری است، اما بسیاری از ابزارها هنوز مسیر (trajectory) اختصاصی خودشان را دارند.

۲. نصب از رابط کاربری

سکوهای وب و فضاهای کاری سازمانی: نصب، بارگذاری و مدیریت پوشه‌ها با چند کلیک از راه یک فهرست بصری، برای کل تیم.

۳. مسیر برنامه‌نویسی

چارچوب‌های سفارشی: مهارت‌ها را برنامه‌نویسی‌شده بار می‌کنید و کد را به مسیر پوشه اشاره می‌دهید.

توصیه: پیش از هر فرضی، مستندات عامل خودتان را بخوانید. قالب مشترک است، اما مسیر نصب، قواعد فعال‌سازی و جزئیات هر ابزار (فهرست ابزارهای مجاز، دروازه‌های (gate) امنیتی، بسته‌بندی افزونه) مشترک نیستند.

مهارت در برابر MCP و AGENTS.md

	چه چیزی است	نسبتش با مهارت
MCP	دربارهٔ **دسترسی** است: عامل را به سامانهٔ بیرونی وصل می‌کند.	رقابت نمی‌کنند، **ترکیب** می‌شوند. وقتی مهارت به داده نیاز دارد، به عامل می‌گوید ابزاری را فراخوانی کند که معمولاً همان سرور MCP فراهمش کرده.
AGENTS.md	همیشه در پروژه بارگذاری شده است.	مهارت‌ها هنگام نیاز بارگذاری می‌شوند. بهترین پیکربندی هر دو را دارد: AGENTS.md را فشرده نگه دارید و در پایانش فهرست کوتاهی از مهارت‌های موجود بگذارید تا نقش مسیریاب کتابخانه را هم بازی کند.

مدل ذهنی یک‌خطی: سیستم پرامپت = غریزه · AGENTS.md = فایل راهنمای پروژه · ابزار و MCP = دست‌ها · بازیابی (retrieval) = کتابخانه · **مهارت = دفترچهٔ راهنمایی که همکار باتجربه روز اول به شما می‌دهد و هوش مصنوعی هرگز فراموشش نمی‌کند.**

کارگاه ۱ — ۲۰ دقیقه

نخستین `SKILL.md` خودتان را بنویسید

۱

یک گردش‌کار تکراری انتخاب کنید

از همان سه موردی که پیش از کلاس نوشتید.

۲

اول توصیف را بنویسید

چه می‌کند، کِی استفاده شود، کِی نشود.

۳

بدنه را بنویسید

کِی به کار می‌رود، کِی نه، گردش کار گام‌به‌گام، یک نمونهٔ ورودی/خروجی.

۴

تفکیک کنید

جزئیات به `references/`، کار قطعی به `scripts/`.

معیار پذیرش: باید بتوانید **سه محرک مثبت و سه محرک منفی** بنویسید. اگر نتوانستید، یا توصیف مبهم است یا مهارت دو کار انجام می‌دهد و باید تجزیه (decomposition) شود.

بخش دو

چرا این‌قدر سریع فراگیر شدند؟

یک مثال واقعی، بهتر از هر استدلالی نشان می‌دهد چه چیزی عوض شد.

مسئله: شرکت لجستیکی با ۱۰۰ گونهٔ فرایند

گونه‌ها بسته به نوع محصول، ابزار، محدودیت مسیر، تعهد سطح خدمت مشتری و مناطق مقرراتی فرق می‌کنند. چهار راه‌حل ممکن:

راه‌حل	چه می‌شود
یک عامل، یک پنجرهٔ غول‌آسا	پوسیدگی فوری زمینه (context) و هزینهٔ توکن سرسام‌آور.
بازیابی روی دفترچه‌ها	دو سال پیش احتمالاً پاسخ درست بود. حالا یعنی نگه‌داری پایگاه بُرداری، مدل امبدینگ (embedding) و راهبرد تکه‌تکه‌سازی — که کیفیتش هیچ ربطی به خود رویه‌ها ندارد.
چندعاملی، هر فرایند یک زیرعامل (subagent)	صد دیپلوی، صد سطح ارزیابی (evaluation) و لایه‌های مسیریابی (routing) پیچیده. کابوس عملیاتی.
یک عامل، صد مهارت	افشای تدریجی یعنی حدود ۵,۰۰۰ توکن فرادادهٔ همیشه‌بارگذاری‌شده. رویه‌ها در کنترل نسخه‌اند و افزودن گونهٔ صدویکم، یک پوشهٔ تازه است نه یک دیپلوی (deploy) تازه.

اما مهارت‌ها معماری چندعاملی را نمی‌کشند

چندعاملی هنوز پاسخ کاملاً درست است وقتی که:

موازی‌سازی واقعی

کارها واقعاً هم‌رانتایم می‌شوند.

مرز واقعی توانمندی

دسترسی متفاوت، وضعیت امنیتی متفاوت، سامانهٔ بیرونی متفاوت.

تجزیهٔ سلسله‌مراتبی

لایه‌های انتزاع واقعاً با هم فرق دارند.

ساختار وارسی متقابل

آرایش تخاصمی یا کنترل و توازن.

ارتباط میان زیرعامل‌ها

زیرعامل‌ها باید با هم حرف بزنند.

مدل‌های ناهمگون

هر بخش مدل متفاوتی می‌خواهد.

آنچه مهارت‌ها کردند: یک اولیهٔ معماریِ گمشده را اضافه کردند. بسیاری از سامانه‌هایی که **به‌طور پیش‌فرض** چندعاملی ساخته می‌شدند، حالا می‌توانند **به‌طور طراحی‌شده** به «یک عامل با مهارت» ساده شوند.

چرا پذیرش این‌قدر سریع بود؟

> پذیرش از مسیر کم‌مقاومت‌ترین راه می‌آید. هرکسی که بتواند مستند بنویسد، می‌تواند مهارت بنویسد.

و همین، سد ورود را پایین می‌آورد: دانش رویه‌ای نهفته‌ای که در ویکی‌ها، دفترچه‌ها و ذهن مهندسان مانده بود، بالاخره جایی ساختارمند برای جاری‌شدن پیدا می‌کند.

بخش سه

ارزیابی مهارت‌ها

حالا یک مهارت دارید، یا شاید کتابخانهٔ کوچکی. پرسش فوری: از کجا می‌دانید واقعاً کار می‌کنند؟

مهارتی که تست (test) ندارد، یک قابلیت نیست؛ یک امید است.

در بنچمارکی روی ۸۴ کار واقعی، ۱۹٪ موارد **با** مهارت بدتر از بدون مهارت عمل کردند — مهارت‌های بدطراحی فقط نویز خنثی نیستند، فعالانه توانایی را کم می‌کنند.

چهار حالت شکست

۱

شکست محرک

مهارت اشتباه فعال می‌شود، یا مهارت درست فعال نمی‌شود.

کجا دیده می‌شود: گزارش‌های مسیریابی.

۲

شکست اجرا

مهارت درست فعال می‌شود اما خروجی نادرست یا فراخوان ابزار (tool call) خطا تولید می‌کند.

کجا دیده می‌شود: کیفیت خروجی.

۳

شکست بودجهٔ توکن

بدنهٔ حجیم مهارت کانتکست ویندو (context window) را می‌بلعد و عملکرد را در نوبت‌های **بی‌ارتباط** خراب می‌کند.

کجا دیده می‌شود: زیر بار واقعی زمینه.

۴

پس‌رفت

مهارت تازه با مهارت موجود هم‌پوشانی پیدا می‌کند و مسیریابیِ درستِ قبلی را می‌شکند.

کجا دیده می‌شود: فقط وقتی کل کتابخانه با هم آزموده شود.

جعبه‌ابزار ارزیابی: پنج الگوی مکمل

الگو	چیست	چه شکستی را می‌گیرد
ارزیابی به‌مثابه تست واحد	فایل تست مهارت که در هر تغییر روی سی‌آی (CI) اجرا می‌شود؛ تست مردود، ادغام را مسدود می‌کند.	همه — برای هر مهارت و هر تغییر
مجموعه‌دادهٔ طلایی	جفت‌های گزینش‌شده و نسخه‌بندی‌شدهٔ ورودی و خروجی مورد انتظار، ذخیره‌شده کنار خود مهارت.	اجرا و محرک — از سطح پیش‌نویس به بالا
داور مدل‌زبانی (LLM-as-Judge)	مدلی هم‌رده خروجی را در مقیاس با سنجهٔ مشخص نمره می‌دهد.	اجرا — پیش از رسیدن به سطح کنش
تخاصمی / تیم قرمز	کاوش نظام‌مند برای بیرون‌کشیدن حالت‌های شکست: برای هر محرک مثبت، یک بازنویسی و یک حالت مرزی (edge case) منفی.	محرک و اجرا
قناری / سایه	دیپلوی روی ترافیک کنترل‌شده پیش از انتشار کامل: مقایسهٔ موازی برون‌خط، یا یک درصد از ترافیک زنده زیر پایش.	پس‌رفت (regression)

محرک، نخستین دروازه است

مهارتی که هرگز فعال نمی‌شود نمی‌تواند کمک کند. مهارتی که بیش‌ازحد گسترده فعال می‌شود، زمینهٔ نامربوط تزریق می‌کند.

شواهد میدانی

در یک تحلیل تولیدی، مهارت‌هایی که انتظار می‌رفت پیوسته فعال شوند، **۵۶٪ نرخ عدم فعال‌شدن** داشتند.

و بدتر

مهارتی که دستورهایش حذف شده بود امتیاز ۵۸ گرفت، در حالی که عامل **بدون** مهارت ۶۳ گرفت. این کسری پنج‌واحدی یعنی مهارت بد، توانایی را کم می‌کند.

و نکتهٔ مکمل: در همان مطالعه، یک فهرست منفعل از قراردادهای پروژه در `AGENTS.md` نرخ قبولی ۱۰۰٪ در برابر پایهٔ ۵۳٪ گرفت. یعنی مهارت‌ها برای گردش‌کارهای **باریک و کنش‌محور** مناسب‌اند؛ زمینهٔ سراسری باید در مستندات همیشه‌دردسترس بماند.

چهار وارسی برای توصیف

برای رسیدن به دقت محرک در حد استاندارد صنعت، توصیف `SKILL.md` — تنها چیزی که مدل هنگام مسیریابی می‌بیند — باید این چهار تست را پاس کند:

ویژگی تست‌پذیر

باید بتوانید سه محرک مثبت و سه محرک منفی بنویسید.

روشنی

پرسش‌های مبهم نباید با مهارت‌های همسایه هم‌پوشانی پیدا کنند.

صداقت اجرایی

عملکرد واقعی را توصیف کند، نه رفتار آرمانی.

پایداری در بازنویسی

مستقل از اینکه کاربر چطور جمله را بسازد، یکسان مسیریابی کند.

خروجی نهایی در برابر مسیر ابزار

وقتی مهارت فعال شد، هر دو را جداگانه بسنجید: آنچه عامل می‌گوید، و آنچه عامل انجام می‌دهد.

شکاف قابل‌توجه: تحلیل‌ها نشان می‌دهد نمره‌دهی فقط بر خروجی نهایی، ۲۰ تا ۴۰ درصد بیشتر از نمره‌دهی مسیرآگاه قبول می‌کند. این شکاف یعنی مواردی که عامل با **توالی نادرست** فراخوان ابزار به پاسخ درست رسیده است.

سطح مهارت	سخت‌گیری لازم در مسیر
فقط‌خواندنی	ترتیب دلخواه کافی است؛ رسیدن به پاسخ درست مهم است.
مجاز به کنش	ترتیب دقیق یا زیرمجموعهٔ مرتب لازم است؛ مسیر نادرست می‌تواند عوارض برگشت‌ناپذیر بسازد.

توسعهٔ ارزیابی‌محور: اول تست، بعد مهارت

```
{
  "case_id": "refund_dup_charge_001",
  "input": "I was charged twice for order #4521 last Tuesday",
  "expected_skill": "refund_processor",
  "expected_tool_calls": [
    {"tool": "lookup_order", "args": {"order_id": "4521"}},
    {"tool": "check_duplicate_charge", "args": {"order_id": "4521"}}
  ],
  "expected_output_format": "confirmation_with_refund_id",
  "rubric": ["acknowledges duplicate", "cites order id", "provides next step"]
}
```

چرا این کار می‌کند: نوشتن سه پروندهٔ این‌چنینی پیش از نوشتن بدنه، ابهام‌های توصیف و خطاهای مسیر ابزار (Tool trajectory) را زودتر بیرون می‌کشد — پیش از آنکه در بدنهٔ مهارت انباشته شوند.

دام ارزیابی منزوی

هرگز مهارت را فقط به‌تنهایی ارزیابی نکنید. عامل‌ها در تولید، پنج تا پانزده مهارت را هم‌زمان بارگذاری می‌کنند.

چه اتفاقی می‌افتد

بدنه‌ای که از پنج هزار توکن بیشتر می‌شود ممکن است تنها عالی کار کند، اما هم‌زمان با بقیه، پوسیدگی زمینه (Context rot) می‌سازد. پژوهش‌ها افت دقت چشمگیر ناشی از تکثیر ابزار و رقابت توجه را ثبت کرده‌اند.

چه باید کرد

برای کالیبراسیون، مهارت را با الگوی «زیرعامل تک‌مهارتی» بسنجید (عامل + یک مهارت در برابر عامل پایه) و هم‌بارگذاری چندمهارتی را برای مرحلهٔ آماده‌سازی تولید نگه دارید.

هشدار توهم (hallucination) ارزیابی: تست مسیر، سامانهٔ مرکب «عامل میزبان + مهارت» را می‌سنجد، نه مهارت را در انزوا. پس منطق اجرا را از مسیریابی جدا کنید: کد ابزار را مستقل اعتبارسنجی کنید و محرک‌ها را روی چند خانوادهٔ مدل ممیزی کنید تا توصیف‌های شکننده و قفل‌شده به یک معماری را بگیرید.

نردبان اختیار: مهارت باید پله‌به‌پله ارتقا یابد

سطح	شرط عبور
فقط‌خواندنی	ارزیابی با داور (judge) مدل‌زبانی؛ دقت محرک در حد هدف.
فقط‌پیش‌نویس با بازبینی انسانی	مجموعه‌دادهٔ طلایی با دست‌کم ۲۰ پرونده؛ تأیید انسانی.
مجاز به کنش	تیم قرمز کامل؛ موفقیت **پایدار** در چند اجرا (نه یک قبولی خوش‌شانس)؛ بدون هیچ رویداد رول‌بک (rollback).

سنجهٔ pass^k: ارزیابی را k بار اجرا کنید و موفقیت در **همهٔ** اجراها را لازم بدانید. در یک بنچمارک (benchmark) شناخته‌شده، مدلی که در اجرای اول ۶۱٪ گرفت، در pass^8 به زیر ۲۵٪ سقوط کرد. موفقیت تک‌اجرا، پیش‌بینی‌کنندهٔ بدی برای اتکاپذیری تولید است.

کارگاه ۲ — ۱۵ دقیقه

سه پروندهٔ ارزیابی برای مهارت خودتان

- دو پروندهٔ **مثبت**: ورودی‌هایی که باید مهارت را فعال کنند.
- یک پروندهٔ **منفی**: ورودی‌ای که نباید فعالش کند اما وسوسه‌انگیز است.
- هر پرونده: ورودی، مهارت مورد انتظار، فراخوان‌های ابزار مورد انتظار، قالب خروجی، سنجهٔ نمره‌دهی.

وارسی زوجی: پرونده‌های همدیگر را بخوانید و فقط یک پرسش بپرسید — «آیا بدون خواندن بدنهٔ مهارت، از روی این پرونده‌ها می‌فهمم این مهارت چه کار می‌کند؟» اگر پاسخ منفی است، مشخصات (spec) هنوز روشن نیست.

بخش چهار

از نمونهٔ اولیه تا تولید

خلاصه‌اش این است: مدل دیگر بخش جالب ماجرا نیست، و مهارت‌ها همان ابزار بنیادی مهندسی‌اند که اجازه می‌دهند با اطمینان منتشر کنید.

داخل رانتایم یک عامل چه چیزی هست؟

٪۹۸.۴

از کدبیس یک عامل کدنویسی شناخته‌شده، **زیرساخت عملیاتی** است: دسته‌بندهای مجوز، خطوط لولهٔ فشرده‌سازی زمینه، واگذاری (handoff) به زیرعامل، ذخیره‌سازی نشست (session).

٪۱.۶

خودِ حلقهٔ عامل. مدل پشت یک رابط راه دور نشسته؛ آنچه سامانه را در سطح تولید نگه می‌دارد، مهندسیِ پیرامون آن است.

بینش معماری پشت همه‌چیز: با هم‌گرایی مدل‌های پایه در استدلال پایه، تمایزدهندهٔ اتکاپذیری خودگردان، **مهندسی قطعیِ پیرامون مدل** می‌شود — و درون آن مهندسی، واحدی که ترکیب و بازاستفاده می‌شود، مهارت است.

چرا مهارت، واحد بهبود است؟

نظریهٔ ساده‌انگارانه می‌گوید مدل بهتر، عامل بهتر می‌سازد. در تولید، مدل زیرساخت است و مهارت‌ها اولیه‌ای‌اند که اجازه می‌دهند بهبود منتشر شود.

مشروط‌اند

فقط وقتی بارگذاری می‌شوند که توصیفشان با کار جاری بخواند.

ترکیب‌پذیرند

یک مهارت می‌تواند ابزار مهارت دیگر را فراخوانی کند یا در زنجیره قرار بگیرد، بی‌آنکه هیچ‌کدام از دیگری خبر داشته باشد.

مالک دارند

هرکدام در پوشه‌ای نسخه‌بندی‌شده با نویسندهٔ مشخص قرار دارد؛ پس بهبود توزیع می‌شود و در تیم مرکزی گلوگاه نمی‌سازد.

چهار سبک بهبود، در یک نگاه

سبک بهبود	زمان چرخه	حالت شکست	چه کسی می‌تواند
تعویض مدل	روزها تا هفته‌ها	پس‌رفت در کارهای بی‌ارتباط	تیم یادگیری ماشین یا سکو
ویرایش سیستم پرامپت	دقیقه‌ها تا ساعت‌ها	پوسیدگی زمینه، تعارض دستورها هزینهٔ زمینه: ایستا — در هر نوبت پرداخت می‌شود	هرکس مالک فایل پرامپت است
فاین‌تیون (fine-tuning)	هفته‌ها تا ماه‌ها	فراموشی فاجعه‌بار، بیش‌برازش	فقط تیم یادگیری ماشین
مهارت تازه	ساعت‌ها تا روزها	کران‌دار: فقط نوبت‌های متناظر هزینهٔ زمینه: پویا (dynamic) — فقط هنگام فعال‌شدن	هر تیم حوزه‌ای

حالت شکستی که نمایش‌ها را می‌شکند

شایع‌ترین حالت شکست عامل‌ها در تولید، توهم نیست؛ **سرریز زمینه** است: مدل زمینه‌ای بیش از آنچه بتواند مؤثر به کار ببرد دریافت می‌کند و **بی‌صدا** افت می‌کند، پیش از آنکه اپراتور متوجه شود.

گم‌شدن در میانه

عملکرد وقتی بیشترین است که اطلاعات مرتبط در ابتدا یا انتهای ورودی باشد و در میانه افت می‌کند؛ منحنی‌ای که حتی برای مدل‌های آموزش‌دیده روی زمینهٔ بلند هم برقرار است.

پوسیدگی زمینه

در بررسی ۱۸ مدل پیشرو، عملکرد با بزرگ‌شدن ورودی افت می‌کند حتی وقتی دشواری کار ثابت نگه داشته شود — و سریع‌تر وقتی محتوای مرتبط از مزاحم‌ها سخت تفکیک شود.

و بدترین نوع نویز: دقیقاً همان چیزی است که زمینهٔ واقعی عامل‌ها پر از آن است — خروجی ابزارها، بازیابی‌های نیمه‌مرتبط و استدلال‌های میانی.

ریاضیات بودجهٔ توکن

عاملی با پنجاه گردش‌کار متمایز را در نظر بگیرید:

به‌شکل یک سیستم پرامپت

۱۵٬۰۰۰ توکن در **هر نوبت** بارگذاری می‌شود.

به‌شکل کتابخانهٔ مهارت

حدود **۴٬۰۰۰ توکن** توصیف، به‌علاوهٔ بدنهٔ حدوداً **۲٬۰۰۰ توکنی** تنها مهارت فعال: در مجموع حدود **۶٬۰۰۰ توکن**، و چهل‌ونُه بدنهٔ دیگر روی دیسک.

نمونهٔ منتشرشده: تبدیل یک گردش‌کار به مهارت، زمینهٔ فعال را از حدود ۱۵۰٬۰۰۰ توکن به ۲٬۰۰۰ توکن رساند — بیش از ۹۸ درصد کاهش.

سه پیامد عملی

ظرفیت، سنجهٔ اشتباهی است

پنجره‌ای با ظرفیت یک میلیون توکن می‌تواند در پنجاه هزار توکن افت معناداری نشان دهد.

زمینهٔ فعال یک بودجه است، نه یک ظرف

هر توکن پیش روی مدل، توجه را از هر توکن دیگر می‌گیرد. با سیستم پرامپت همان‌طور رفتار کنید که تیم زیرساخت با حافظه (memory): منبعی متناهی که آگاهانه تخصیص می‌یابد.

مهارت‌ها قید را حل می‌کنند

زمینهٔ فعال را کوچک نگه می‌دارند، در حالی که توانایی در دسترس عملاً بی‌کران (Unbounded) می‌ماند.

بخش پنج

فرامهارت‌ها و مهارت‌های خودبهبوددده

تا اینجا هر مهارت را انسان نوشت. پرسش طبیعی بعدی: آیا عامل می‌تواند در نوشتن، ارزیابی و بهبود مهارت‌ها کمک کند؟

چهار دستهٔ فرامهارت

۱

نگارش

مهارتی که توصیف یک گردش‌کار را می‌گیرد و پیش‌نویس `SKILL.md` تولید می‌کند.

۲

نگارش کمکی از روی رد اجرا

به‌جای آنکه انسان گردش‌کار را توصیف کند، عامل را چند بار در انجام موفق آن تماشا کنید و همان رد را به مهارت تبدیل کنید. کار انسان از نوشتن به **تأیید** جابه‌جا می‌شود.

۳

بهبود

مهارتی که یک مهارت موجود به‌همراه مجموعه‌ای از پرونده‌های ارزیابی مردود را می‌گیرد و ویرایش پیشنهاد می‌دهد.

۴

تکامل کتابخانه

عامل کاری را تمام می‌کند که مهارتی برایش نداشت، متوجه می‌شود مسئله‌ای تکرارشونده را حل کرده، و افزودن مهارتی تازه را پیشنهاد می‌دهد.

کجا از هم می‌پاشد

هستهٔ خطر: فرامهارت‌ها (Meta-skill) فقط وقتی کار می‌کنند که مجموعهٔ ارزیابی شما خوب باشد. عاملی که اجازه دارد مهارت‌های خودش را ویرایش کند، با کمال میل برای هر سنجه‌ای که به آن اشاره کنید بهینه می‌شود — از جمله سنجه‌هایی که به‌آسانی می‌شود سرشان کلاه گذاشت.

- **هر چیزی که عامل می‌نویسد، در سطح پیش‌نویس وارد کتابخانه می‌شود**

  هرقدر هم فرامهارت مطمئن باشد؛ و از همان نردبان مهارت‌های انسان‌نوشته بالا می‌رود.

- **برای چند ویرایش نخست، انسان را در حلقه (loop) نگه دارید**

  حتی وقتی سنجه (rubric) آشکارا بهتر شده، تفاوت‌ها را اسکن کنید. خطای عامل — بیش‌برازش توصیف به چند پروندهٔ تست، خراب‌کردن مهارتی که از وجودش خبر نداشت — دقیقاً از آن نوعی است که انسان در سی ثانیه می‌گیرد.

- **با فرامهارت شروع نکنید**

  سریع‌ترین راه برای ساختن یک کتابخانهٔ بد، این است که عامل را به پوشه‌ای خالی اشاره کنید و بخواهید پنجاه مهارت بسازد.

ترکیب مهارت‌ها: هماهنگی گراف بدون دور

گردش‌کارهای واقعی درون یک مهارت جا نمی‌شوند. رد و بدل خام خروجی مدل میان مهارت‌های منزوی کارساز نیست: وضعیت مبهم می‌شود، اجرا غیرقطعی (not deterministic) می‌شود و دیباگ (debug) سخت.

وضعیت جداشده

مسیریابی وضعیت به انباشتن تاریخچهٔ اجرا در پرامپت مدل تکیه ندارد.

گذرگاه پیام فایلی

کنترل‌گر گراف، تحویل‌ها را با رد و بدل ارجاع‌های اسکیماای میان گره‌ها هماهنگ می‌کند.

توجه محافظت‌شده

جداکردن بار داده از ورودی متنی مدل، از تورم کانتکست ویندو جلوگیری می‌کند و ظرفیت مدل را حفظ می‌کند.

قاعدهٔ طلایی: هرگز از کانتکست ویندوی مدل به‌عنوان پایگاه داده استفاده نکنید. فقط نشانی یا اشاره‌گر را از راه فایل‌سیستم یا گذرگاه پیام رد کنید.

نمایه‌های توانمندی و آرایش‌های معماری

نمایهٔ توانمندی (Capability profile): بسته‌ای قابل‌تعویض که مهارت‌های فعال و دسترسی ابزارها، سیستم پرامپت‌ها و گاردریل‌ها، گردش‌کارهای خودکار، و پارامترهای مدل را با هم تعریف می‌کند. هنگام اجرا، لایهٔ هماهنگی دستورهای پیشین را تخلیه و متغیرهای کهنه را پاک می‌کند و سپس نمایهٔ تازه را جای آن می‌گذارد.

معماری	سازوکار	مناسب چه کاری
خط لولهٔ خطی	رد و بدل متوالی متن میان گره‌های ثابت	کارهای مولد تک‌حوزه‌ای و کم‌پیچیدگی؛ نمونه‌سازی سریع
هماهنگی گراف بدون دور	اجرای موازی گراف‌محور با رد و بدل وضعیت از راه ارجاع اسکیما	گردش‌کارهای چندعاملی که اتکاپذیری بالا می‌خواهند
نمایه‌های توانمندی	بسته‌های قابل‌تعویض و نسخه‌بندی‌شدهٔ پارامتر و ابزار	دیپلوی نقش‌محور و عامل‌های حوزه‌ویژه

بدهی زمینه و انتقال هوش به چپ

بدهی زمینه

وقتی نویسندگان می‌کوشند با متورم‌کردن توصیف‌ها (مثلاً «ALWAYS DO X») رفتار قطعی بسازند، بدهی زمینه (Context debt) انباشته می‌شود. مدل‌ها یاد می‌گیرند این دستورهای پرتأکید را نادیده بگیرند — دقیقاً همان‌طور که یک توسعه‌دهنده دیوار هشدارهای ناخوانا را نادیده می‌گیرد.

انتقال هوش به چپ

به‌جای امید به اینکه مدل قواعد پیچیده را هنگام اجرا درست تفسیر کند، قضاوت‌های ذهنی را در مهارت‌ها تقطیر کنید و منطق را از پرامپت بیرون بکشید و در اسکریپت‌های استاندارد و تست‌پذیر بگذارید.

نرم‌افزار بنویسید، نه قاعده: دستورهای منفی به مدل را با قیدهای نرم‌افزاری قطعی جایگزین کنید که کنش نامعتبر را **ناممکن** می‌کنند.

بخش شش

انتخاب از میان هزاران مهارت

تا اوایل ۲۰۲۶، مارکت‌پلیس‌های (marketplace) عمومی مهارت از ده‌ها هزار فهرست عبور کرده بودند. مسئلهٔ انتخاب، واقعی و رو به رشد است.

سه معیار انتخاب

۱

برای ابزارهای اختصاصی، مهارت دست‌اول را ترجیح دهید

هر چیزی که نویسندگانِ خودِ سامانهٔ زیرین نوشته باشند، درست‌تر و نگه‌داشته‌شده‌تر از جایگزین‌های جامعه است.

۲

هر چیزی را که به آن وابسته‌اید قفل نسخه کنید

مهارت‌های جامعه تکامل می‌یابند و وابستگی (dependency) قفل‌نشده‌ای که دیروز کار می‌کرد، فردا خراب می‌شود.

۳

پیش از پذیرش، ممیزی کنید

مهارت، کدی است که در زمینهٔ شما اجرا می‌شود. با آن مثل هر وابستگی دیگری رفتار کنید، با همان بهداشت زنجیرهٔ تأمین.

سه دستهٔ منبع، سه سیاست

منبع	پیش‌فرض اعتماد	چه کسی نگه‌داری می‌کند
مهارت‌های دست‌اول ارائه‌دهنده	اعتماد پیش‌فرض؛ اما نسخه را قفل کنید	همان تیمی که محصول زیرین را ساخته است
مهارت‌های گزینش‌شدهٔ سازمان	اعتماد درون سازمان؛ بازبینی هنگام پذیرش	تیم‌های حوزه‌ای خودتان، با بازبینی در پول‌ریکوئست (pull request)
مهارت‌های جامعه	پیش از پذیرش ممیزی؛ قفل نسخهٔ سخت‌گیرانه	نویسندگان داوطلب، با تعهد نگه‌داری متغیر

از فردا از کجا شروع کنیم

1. **یک ساعت با باتجربه‌ترین متخصص تیم بنشینید.**

   از او بخواهید سه گردش‌کاری را که مرتب انجام می‌دهد روایت کند. ضبط کنید — این متن، تقریباً به‌معنای واقعی کلمه، پیش‌نویس اول سه مهارت است.

2. **پرتکرارترین را انتخاب کنید و خودتان بدون هیچ مهارتی اجرایش کنید.**

   جایی را که عامل شکست می‌خورد یادداشت کنید. خطای واقعی سیگنال تولید می‌کند؛ گمانه‌زنی نویز.

3. **سه پروندهٔ ارزیابی بنویسید، بعد بدنه را.**

   دو مثبت و یک منفی.

4. **در سطح فقط‌خواندنی منتشر کنید و توصیف را تا رسیدن به دقت هدف اصلاح کنید.**

و مقاومت کنید در برابر وسوسهٔ اینکه از روز اول یک مدل زبانی پنجاه مهارت تولید کند.

جلسهٔ بعد

امنیت و ارزیابی عامل‌ها

عامل را توانمند کردیم؛ حالا می‌پرسیم
چه چیزی ممکن است خراب شود و چطور بسنجیمش

تکلیف: مهارت‌تان را تا رسیدن دقت محرک به حد قابل‌قبول اصلاح کنید.